

Introduction to Artificial Intelligence – Final Project

Lecturer: Mr. Akiva Bruno Malka
Teaching Assistant: Mr. Regev Yehezkel Imra

Submission Date: **March 12, 2026**

1 Project Overview and Submission Guidelines

This final project is composed of three parts, each focusing on a different aspect of supervised learning. The goal of the project is to demonstrate a clear understanding of modeling choices, evaluation methodology, and practical implementation of machine learning algorithms.

1.1 Project Structure

The project consists of the following three parts:

- **Part I: Regression Task**

Analysis of a supervised regression problem using the Auto MPG dataset. This part focuses on model assumptions, bias–variance tradeoff, optimization behavior, and generalization through linear, polynomial, and KNN regression.

- **Part II: Classical Classification Task**

Analysis of a multi-class classification problem using the CIFAR-10 dataset. This part focuses on classical machine learning models (Logistic Regression, SVM, KNN), hyperparameter selection, and model evaluation on high-dimensional image data.

- **Part III: Neural Network Classification (PyTorch)**

Implementation and analysis of a neural network classifier using a provided PyTorch notebook. In this part, you will complete the missing implementation blocks and design a model that achieves strong performance, while carefully justifying all architectural and optimization choices.

1.2 Submission Requirements

The project may be submitted **individually or in pairs**.

You are required to submit:

- An academic-style report written in **Overleaf**
- A **GitHub repository** containing all code used in the project

The report must be written in a clear and organized academic format and must describe all modeling decisions, experiments, and results presented in the code.

1.3 Code and Reproducibility

All code written for this project must be included in the submitted GitHub repository.

- The repository must include all scripts and notebooks used to generate the results reported in the Overleaf document.
- In particular, the PyTorch notebook from Part III, including all outputs and final results, must be uploaded to GitHub.
- The results reported in the Overleaf document must exactly match the results produced by the submitted code.

The Overleaf report must include a link to the GitHub repository containing all code and notebooks used in the project.

Code that does not run or does not reproduce the reported results will result in **disqualification of the submission**.

1.4 Overleaf Submission

Upon submission, you must share the Overleaf project with the instructor and the teaching assistant as **Editors** via email:

`bruno.melka@gmail.com`
`regevel2006@gmail.com`

2 Regression Task: Fuel Efficiency Prediction

In this part of the project, you will analyze a supervised regression problem using the **Auto MPG** dataset. The Auto MPG dataset used in this part is publicly available at: <https://archive.ics.uci.edu/dataset/9/auto+mpg>

The goal is to predict a car's fuel efficiency (measured in miles per gallon, mpg) based on vehicle characteristics.

The purpose of this task is not necessarily to achieve the best possible prediction accuracy, but to demonstrate a clear understanding of modeling assumptions, model complexity, optimization behavior, and generalization.

Throughout this section, **every modeling decision must be justified using both quantitative results and visual evidence (plots)**.

2.1 Exploratory Data Analysis

Before training any regression models, you must perform an exploratory analysis of the dataset.

Your analysis must address the following points:

- Whether the dataset contains missing values, and in which features
- The distribution of the target variable (mpg)
- Relationships between individual features and the target
- Correlations among features

All observations must be supported by appropriate visualizations and discussed in the text.

2.2 Dataset and Preprocessing

You must use the Auto MPG dataset provided with the assignment.

Target variable:

- mpg

Available features include:

- cylinders, displacement, horsepower, weight, acceleration, model_year, origin

You must clearly describe and justify:

- Which features are used for prediction
- How missing values (if any) are handled
- Whether features are standardized, and why

2.3 Linear Regression Baseline

You must begin your analysis with a **linear regression** model.

This model serves as a baseline and should be treated as the simplest reasonable approach to the problem.

Required figures:

- Feature–target relationship plots (e.g., mpg vs weight)
- Prediction vs ground truth plot

You must report the following evaluation metrics on held-out data:

- MSE, RMSE, MAE, and R^2

In the accompanying text, you must explain:

- Whether the linearity assumption appears reasonable
- What limitations or systematic errors are visible in the model predictions

2.4 Polynomial Regression and Model Complexity

Next, you must increase model flexibility using **polynomial regression**.

You are required to:

- Train polynomial regression models of increasing degree
- Clearly describe how the polynomial degree was selected

Required figure:

- Model complexity curve showing training and validation error as a function of polynomial degree

In the text, you must discuss:

- Evidence of underfitting and overfitting
- How increasing model complexity affects bias and variance
- Which polynomial degree was ultimately chosen for the final model, why, and how

2.5 K-Nearest Neighbors (KNN) Regression

You must analyze a **non-parametric regression model** using K-Nearest Neighbors (KNN) regression.

You are required to:

- Evaluate multiple values of k
- Select the best k based on validation performance

Required figures:

- Train and validation error as a function of k
- Prediction vs ground truth plot

You must explain:

- How the choice of k affects bias and variance
- In what situations KNN outperforms or underperforms parametric models

2.6 Optimization Behavior

You must analyze the behavior of different optimization strategies.

Train the same regression model using at least two different optimization settings (e.g., batch gradient descent vs stochastic or mini-batch gradient descent).

Required figure:

- Train and validation loss as a function of iterations or epochs

You must discuss differences in convergence speed, stability, and noise.

2.7 Model Comparison and Discussion

Conclude the regression task with a comparison of all regression models studied (linear, polynomial, and KNN).

You must summarize and compare the models in terms of:

- Predictive performance
- Bias–variance characteristics
- Interpretability
- Practical considerations

You must conclude by stating which model you would choose for deployment and justify your choice. After selecting the final model based on validation performance, you must report its performance on the test set. The test set must not be used for model selection or hyperparameter tuning.

3 Classification task: Classical Classification on CIFAR-10

In this part of the assignment, you will study a supervised multi-class classification problem using the CIFAR-10 dataset.

The dataset can be downloaded from: <https://www.cs.toronto.edu/~kriz/cifar.html>

Under **Version**, you must download the **CIFAR-10 python version**.

The purpose of this part is to analyze how classical machine learning models perform on high-dimensional image data, and to understand their limitations. The CIFAR-10 dataset consists of 32×32 color images belonging to 10 different classes.

3.1 Baseline Models

You are required to implement and evaluate the following classification models:

- Multiclass Logistic Regression (Softmax)
- Linear Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

Each model must be trained using the same train/validation split.

3.2 Hyperparameter Selection

For each model, you must explicitly select and justify at least one hyperparameter. Examples include:

- Regularization strength (Logistic Regression / SVM)
- Number of neighbors k (KNN)

Hyperparameters must be chosen using validation performance. You are required to include plots that support your selection (e.g., validation accuracy as a function of k).

3.3 Evaluation and Analysis

For each model, report:

- Validation accuracy
- Confusion matrix

You must compare the models and discuss:

- Which models perform better and why
- The effect of high input dimensionality
- Limitations of classical models on image data

After comparing the models and selecting a final model using validation performance, you must report the classification accuracy of this model on the test set.

The test set must not be used for model comparison or hyperparameter tuning.

4 PyTorch task: Neural Network Classification

In this part of the assignment, you will work with the provided Python notebook (PyTorchNN.ipynb). The notebook already contains the data loading, training loop, and evaluation code.

Your task is not to rewrite the notebook. Instead, you must read and understand the code that is already provided, and then complete the missing implementation marked with `TODO`.

4.1 What you must implement

Inside the `TODO` block(s), you are required to implement your model and training configuration in order to achieve the best performance you can on the given dataset.

This includes choosing, at minimum:

- The model architecture (e.g., number of layers, hidden sizes, activation functions)
- Optimization choices (e.g., optimizer type, learning rate, batch size)
- Regularization choices if used (e.g., weight decay, dropout, early stopping)

4.2 How you must choose hyperparameters

You are **not allowed** to select hyperparameters by guessing. You must write code in the notebook that supports your choices (for example: a small search over hyperparameters, validation-based selection, or a clear comparison between several alternatives).

Your submitted notebook must include the code you wrote to perform this selection process, along with the final configuration you chose.

4.3 What to write in the report

In your Overleaf report, you must describe and justify:

- How you designed your architecture and why
- How you selected hyperparameters (what you compared, what criterion you used)
- Which configuration performed best and how you verified it
- What you learned from the comparisons (not only the final score)

5 Bonus Task (Optional): Neural Network Regression with PyTorch (10 Points)

This bonus task is optional and can earn up to **10 additional points**.

In this task, you will adapt the provided PyTorch notebook from Part III (neural network classification) to solve the regression problem from Part I using the Auto MPG dataset.

5.1 Task Description

You are required to:

- Duplicate the PyTorch notebook used in Part III
- Modify it to perform **regression** instead of classification
- Train a neural network to predict the target variable `mpg`

You may reuse as much of the existing code as possible, but you must adapt:

- The model output layer
- The loss function
- The evaluation procedure

5.2 Evaluation and Reporting

Model selection and hyperparameter tuning must be performed using a validation set. After selecting a final model, you must report its performance on a held-out test set.

In your Overleaf report, you must describe:

- The architecture of the neural network and how it differs from the classification model
- The loss function used for regression and why it was chosen
- A comparison between the neural network regression model and the best regression model from Part I

Required figures (at least):

- Ground truth vs. predicted values plot
- Training and validation loss as a function of epochs

These figures must be used to support the discussion and conclusions in the report.

5.3 Submission Requirements

If you choose to complete this bonus task:

- The adapted PyTorch regression notebook must be included in your GitHub repository
- The results reported in the Overleaf document must be fully reproducible from the submitted code